Machine learning for fault diagnosis from cluster telemetry

What a learned classifier adds to the rule set, how to build one honestly on the simulator's data, and what it will take to move it onto a real fleet. Every number here comes from a prototype you can re-run.

90
SIMULATED RUNS, 5 SEEDS

1,593
LABELLED WINDOWS

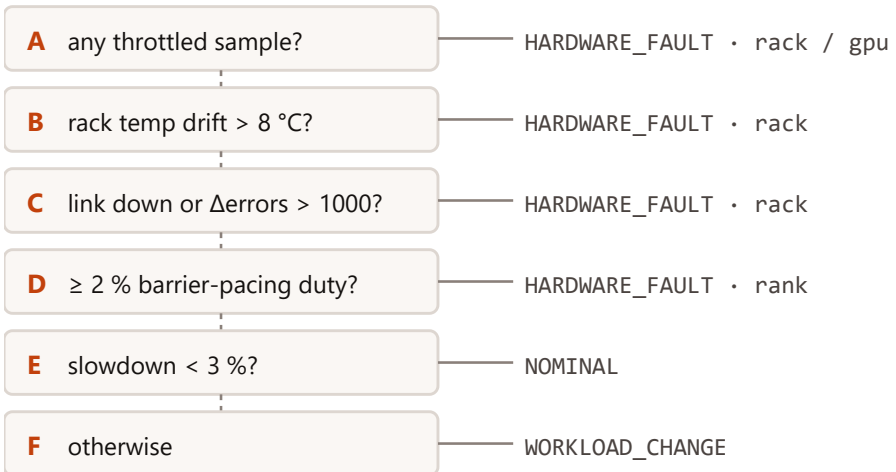
0.997
MACRO-F1, HELD-OUT SEEDS

36/36
HELD-OUT RUNS CLASSIFIED

WHY GO BEYOND RULES

The rule set works, and that is exactly its limit

`metrics.diagnose` scores 18/18 on the matrix it was written against. It is a ladder of thresholds tuned on one seed with fixed fault targets and fixed onsets.



first match returns · 7 hand-set constants · one verdict per run, after the last 30 %

On 36 held-out runs with **re-drawn targets and onsets** the rules still score **36/36**, the learned model **36/36** (slide 7). Six clean signatures are easy for both.

What learning adds

The **joint signature** across tables, a **score per window** as the run progresses, a **ranked list** of suspects, and an operating point the operator chooses.

Fixed thresholds, one operating point

8 °C of rack drift, 1000 frames, 2 % pacing duty, 3 % slowdown. Each was set by looking at one run per scenario. There is no way to trade recall for false alarms, and no calibration to a fleet.

One channel at a time, in a fixed order

A gate reads one table. Nothing combines halo dispersion with uplink drops, or occupancy with clock. Three of nine tables (`nic`, `switch_aggregate`, `events`) are never read.

Verdict at the end, not during

Four gates compare the first 15 % with the last 30 % of the run. A fault present from the start, or one that begins in the last quarter, is misread. There is no early warning and no per-entity ranking.

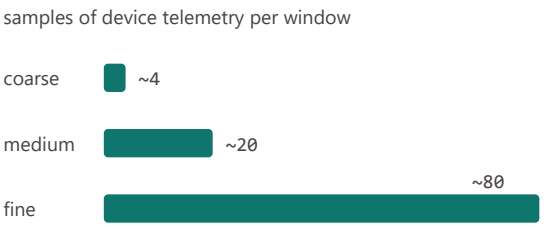
THE DATA

Nine tables on two clocks

Device telemetry is sampled at 1 Hz on wall time; job timing is per iteration. They only meet through `job_performance.timestamp`, and the iteration clock runs 20× faster on the coarse mesh than on the fine one.

Table	Grain	Rows	What it carries
`telemetry_gpu`	GPU × 1 s	24 k	occupancy, utilisation, power, temperature, clock, throttle flag & reason
`telemetry_nic`	NIC × 1 s	6 k	Gbps, cumulative bytes / errors / drops
`telemetry_switch_port`	port × 1 s	18 k	link up, utilisation, queue depth, cumulative counters; leaf ports name their rack
`telemetry_switch_aggregate`	switch × 1 s	1 k	uplink utilisation, oversubscription ratio, congested flag
`telemetry_storage`	backend × 1 s	0.2 k	read / write latency, throughput, dirty backlog
`telemetry_node`	node × 1 s	3 k	cpu / memory / io pressure
`rank_performance`	rank × iteration	128 k	compute, halo, all-reduce wait, checkpoint per rank. **No timestamp.**
`job_performance`	iteration	1 k	iteration time, rank spread, phase means and maxima, **timestamp**
`events`	event	5 k	injections, throttle and congestion transitions. **Labels only, never features.**

A window is 100 iterations



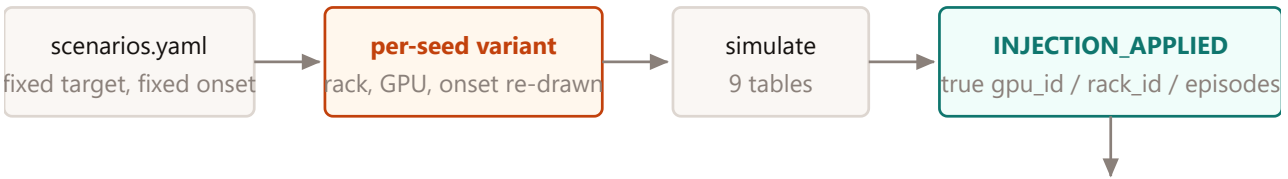
Defining windows in **iterations**, not seconds, keeps the job-timing features comparable across meshes. Every feature must then be count-invariant: means, maxima, fractions and last−first deltas, never sums.

Sample-level jitter is real: a 1 Hz tick lands mid-timestep and is credited pro rata. Anything shorter than a second is invisible in device telemetry, which is why `rank_performance` matters.

LABELS

Ground truth comes from the injector, and seeds are the augmentation axis

The label of a window is whether the fault **physically exists** in it. That is read back from the `INJECTION_APPLIED` payload, which names the real target GPU or rack and, for the straggler, the whole episode schedule.



window labels along one faulted run

straggler: a window is positive if any episode overlaps it. Straddling windows are dropped from training and scoring.

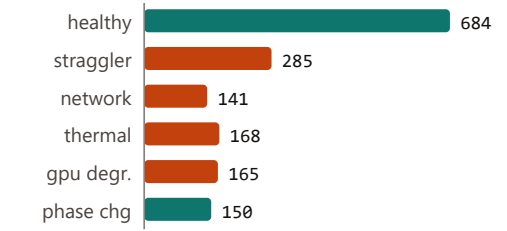
Why re-draw targets

With the yaml's fixed targets, "rack 1" means thermal and "iteration 300" means network. A model would learn the label, not the physics. Per-seed variants are built in memory; the simulator is untouched.

Why seeds, not noise

Seeds key every stochastic stream by entity identity, so each seed relocates the straggler cohort and moves the onset. That is augmentation with physically consistent samples.

labelled windows by class (1,593)



90 runs · seeds 1, 2, 3, 4, 42 · test seeds **3, 4**. Healthy dominates because every pre-onset window is healthy. The classifier is class-weighted and all scores are macro-averaged.

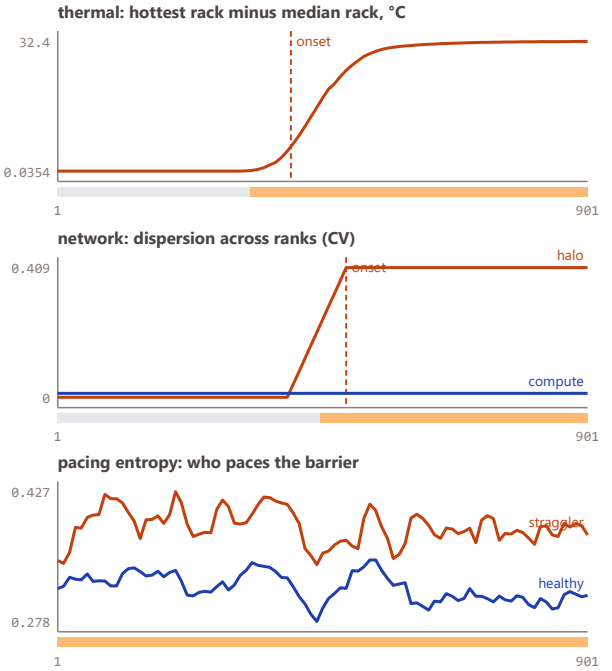
Orange = fault classes. `phase_change` is a labelled non-fault: a 10 % slowdown with clean hardware.

FEATURE ENGINEERING

Make the physics dimensionless

50 window features. Every one is a ratio, a fraction, a dispersion or a deviation from the fleet or rack median, so a single model covers three meshes whose iteration times differ 20×.

Source	Window features (examples)	Which fault they separate
`job_performance`	iteration CV and slope, spread / iteration time, halo / compute, max / mean of each phase, output duty	impact, phase change
`rank_performance`	**pacing duty** of the top rank and its entropy, max mean excess, p99 excess, halo and compute dispersion across ranks, min wait	straggler, gpu degradation, network
`telemetry_gpu`	occupancy, util−occupancy gap, min occupancy / median, min clock / median, power CV, **rack temperature drift** and rise, throttle fractions by reason	thermal, gpu degradation
`nic`, `switch_port`, `aggregate`	error and drop rates per GB from counter deltas, downed-uplink fraction per rack, max queue, max oversubscription, congested fraction	network
`storage`, `node`	write latency mean and max, dirty backlog, storage active fraction, io / cpu / memory	phase change



Held-out medium-mesh runs. Bar under each chart: model's per-window prediction (grey healthy, orange fault).

LEAKAGE

What the model must not see

A simulator hands you perfect labels and perfect identities. Most of the ways to cheat are accidental.

Column(s)	Decision	Reason
`scenario`, `seed`, `summary.json`, the `events` table	**excluded**	the label and its proxies; `events` is read only to build labels
`is_straggler`, `straggler_count`, `THROTTLE_ENGAGED`, `CONGESTION_ONSET`	**excluded**	detector outputs baked into the simulator; pacing duty is recomputed from raw per-rank times
`rank_id`, `gpu_id`, `rack_id`, `slowest_rank_id`, port and switch ids	**excluded**	identity. Used only as join keys; everything entity-level is a deviation from a median
`iteration`, `timestamp`, window position	**excluded**	position in the run; a streaming detector must not depend on it
absolute iteration time, compute time, memory used	**ablation only**	constant within a run: a mesh identifier (slide 8)
`throttled`, `throttle_reason`	**kept**, ablated	the device's own self-report (DCGM exposes it), a consequence not a label; the `-throttle` ablation shows thermal is still caught from rack drift
the healthy twin at the same seed	**never**	same silicon draw; a paired difference is an oracle no fleet has

0.14

MACRO-F1 WITH PERMUTED LABELS

The sanity check: shuffle the training labels, retrain, score the held-out seeds. Chance for six classes is about 0.17. A number well above that would mean a leak survived the table on the left.

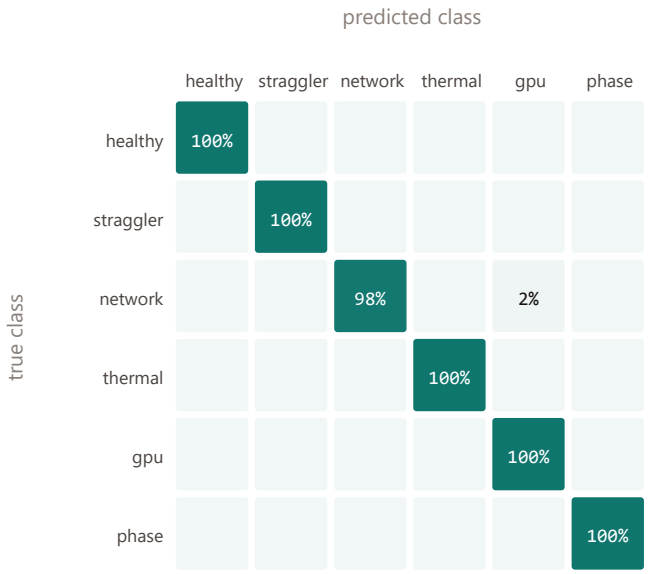
Split by seed, never by row

Windows overlap and neighbours are near-copies. A random row split scores near 1.0 and means nothing. All results here hold out whole seeds (3, 4), and the mesh hold-out holds out whole workloads.

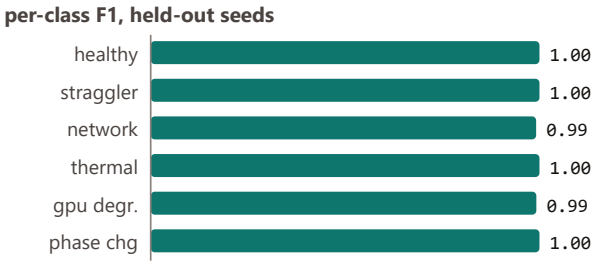
TASK 1 · CLASSIFICATION

Six classes on seeds the model never saw

Gradient-boosted trees on 50 window features, trained on seeds 1, 2, 42, scored on 639 windows from seeds 3, 4.



Row-normalised. Macro-F1 **1.00**, accuracy 100%; 5-fold leave-one-seed-out 0.93 ± 0.12. Fault recall 100%, false-alarm rate on healthy + phase-change windows 0.0%.



Logistic regression on the same features: macro-F1 1.00. The non-linearity buys +0.00.

Run-level verdicts, same held-out runs

Scenario	ML	Rules
healthy	6/6	6/6
straggler	6/6	6/6
network	6/6	6/6
thermal	6/6	6/6
gpu degr.	6/6	6/6
phase chg	6/6	6/6
All	**36/36**	**36/36**

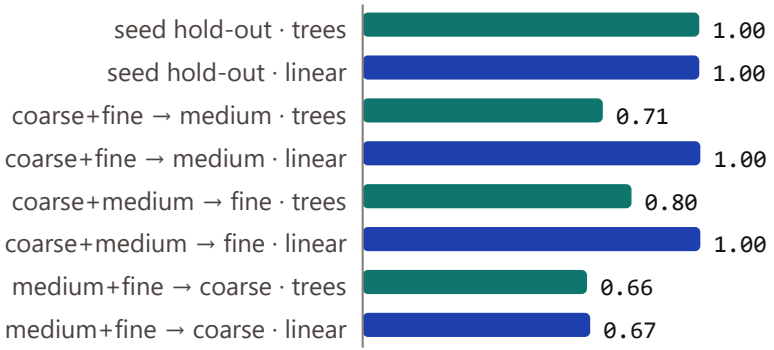
ML verdict = most frequent non-healthy class if it appears in ≥ 3 windows (2 → 100%, 5 → 100%). Rules = `diagnose()` mapped onto the same six classes.

GENERALISATION

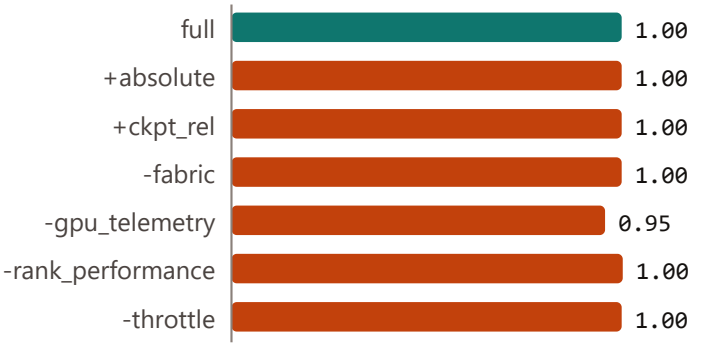
Does it transfer to a workload it has not seen?

Holding out a whole mesh is the closest thing to a domain-shift test the simulator offers: a different cell count, a different iteration time, a different number of samples per window.

macro-F1 by hold-out: gradient-boosted trees vs logistic regression



ablations, seed hold-out



Trees do not extrapolate

In-distribution the two models tie. Hold out a mesh and the trees fall to 0.66–0.80 while the linear model keeps 0.67–1.00 on two of three: a split threshold learned inside one mesh's healthy range says nothing about values outside it, whereas a linear boundary on dimensionless features extends. Recommendation: a linear or monotone-constrained model as the fleet default, trees as the in-distribution refinement.

Removing any one table costs almost nothing: the signatures are redundant across tables. Adding absolute iteration time or the checkpoint-to-iteration ratio does not help in-distribution, and both are mesh identifiers that broke the first mesh hold-out.

Hardest transfer: medium+fine → coarse

Both models struggle (trees 0.66, linear 0.67); healthy F1 0.00. The coarse mesh carries a genuine 4 % load imbalance on healthy hardware, so "one rank always paces the barrier" is normal there and a straggler elsewhere. A fleet has the same problem between applications.

EARLY WARNING

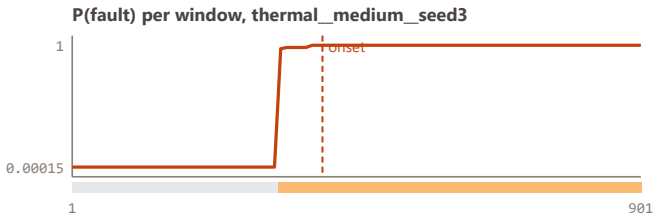
Time to detect, on trailing windows

The same model scores every 10 iterations. Detection = the first window ending after the onset whose class is right for two consecutive windows. The rule set answers once, after the run.

Scenario	coarse	medium	fine
thermal	2/2 · 50 it · 2 s	2/2 · 35 it · 7 s	2/2 · 25 it · 20 s
network	2/2 · 50 it · 2 s	2/2 · 60 it · 14 s	2/2 · 60 it · 53 s
gpu degr.	2/2 · 106 it · 5 s	2/2 · 106 it · 25 s	2/2 · 106 it · 103 s
phase chg	2/2 · 63 it · 3 s	2/2 · 63 it · 13 s	2/2 · 63 it · 54 s
straggler	2/2 · 100 it · 4 s	2/2 · 100 it · 22 s	2/2 · 100 it · 88 s

Cells: detected / runs · median latency after onset in iterations · in wall-clock seconds. Straggler latency is measured from the first iteration, since its episodes start at once. Thermal latency includes physics: the cooling fault ramps over 60 iterations and the die follows with a 20 s time constant.

False-alarm rate per window on held-out runs: healthy **0.0%**, phase change **0.0%**.



Operating point is a choice

P(fault) is a score. Two consecutive windows is one policy; a fleet would tune persistence and threshold against its own false-alarm budget, which a rule ladder cannot offer.

TASK 2 · LOCALISATION

Which GPU, which rack: scoring entities, not runs

A second model scores every (window, GPU) and (window, rack) from fleet-relative features, with the injected entity as label. Consumed as a ranked list, so the metrics are ranking metrics.

Per-window ranking, held-out seeds	supervised GBT	isolation forest	PCA residual
straggler (114 windows)	1.00 / 1.00	0.80 / 1.00	1.00 / 1.00
gpu degr. (63 windows)	1.00 / 1.00	1.00 / 1.00	1.00 / 1.00
thermal (72 windows)	1.00 / 1.00	1.00 / 0.93	1.00 / 1.00
thermal (rack, top-1)	1.00	1.00	1.00
network (rack, top-1)	1.00	0.00	1.00

GPU rows: precision@1 / mean AUROC per window; rack rows: top-1 accuracy. Unsupervised scores are fitted on healthy rows only. The isolation forest ranks the failed rack first **0.00** of the time: downed-uplink fraction and drop rate are constant in healthy data, so no tree ever splits on them and an out-of-range value cannot be isolated. A reconstruction residual has no such blind spot.

Run-level: named the right entity	ML	Rules
straggler	6/6	6/6
gpu degr.	6/6	6/6
thermal	6/6	6/6
network	6/6	6/6

One straggler window, ranked

straggler__coarse__seed3, iterations 451–551, 2 GPU(s) in episode

1	r0n2g1	1.00	← in cohort
2	r2n2g2	1.00	← in cohort
3	r0n1g0	0.00	
4	r0n3g1	0.00	
5	r0n2g3	0.00	
6	r0n2g4	0.00	

Cannot be localised, by construction

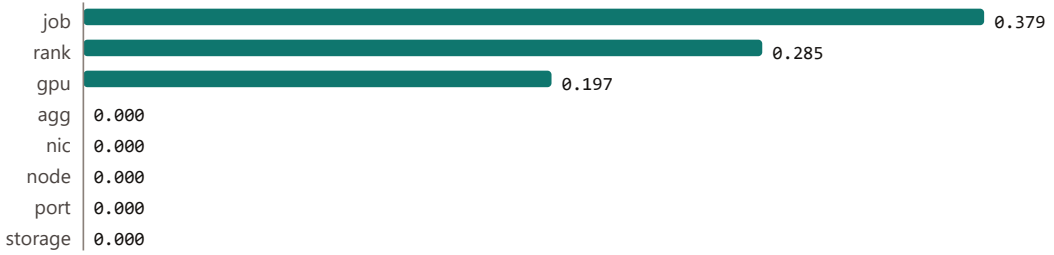
A degraded-but-up uplink shares its rack's counters with its healthy siblings in this model. No feature can rank ports the data does not distinguish. Real leaf switches count errors per PHY, so the first thing a fleet version needs is per-port telemetry asymmetry.

INTERPRETATION

What it learned, and what it ignored

Permutation importance on the held-out seeds: shuffle one feature, measure the F1 it costs. Grouped by source table, then the top two features per class.

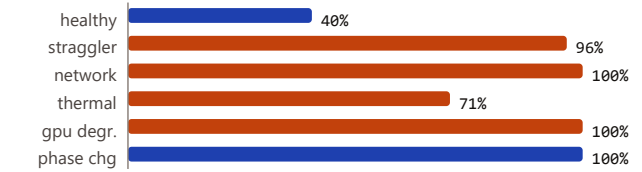
macro-F1 drop when shuffled, summed per source table



Class	Most important features
healthy	`job_output_duty`, `gpu_rack_temp_drift`
straggler	`rank_wait_min_rel`, `rank_pace_top1`
network	`job_halo_max_rel`, `job_output_duty`
thermal	`gpu_rack_temp_drift`, `rank_wait_min_rel`
gpu degr.	`rank_pace_top1`, `rank_wait_min_rel`
phase chg	`job_output_duty`, `gpu_rack_temp_drift`

Unsupervised is-fault AUROC on the same held-out windows: isolation forest 0.68, PCA reconstruction residual 0.96.

isolation forest: windows flagged, by true class



It never touched the fabric counters

Every uplink, NIC and storage feature scores zero: halo dispersion across ranks already names the network fault, output duty the phase change. On a fleet the job-timing tables are the ones most often missing, so the `-rank_performance` ablation matters more.

Novelty is not fault

Fitted on healthy windows, the isolation forest flags **100%** of output-campaign windows: the storage channel moved, so the window is novel. Only labelled non-faults tell the two apart.

SIM TO REAL

Getting this onto a fleet

Train on simulation only after calibration, validate features against real telemetry before validating labels, then fine-tune on real incidents with the simulator as pre-training.

1 · Calibrate, regenerate, retrain

Fit the simulator's constants from what a fleet already records: DCGM power and clock curves, thermal step response, nccl-tests bandwidth and latency, a CRAC derate test. Then regenerate the matrix and retrain; the pipeline is one command.

2 · Validate features before verdicts

Before any label exists, check real telemetry obeys what the features assume: phases sum to the iteration time, counters are monotone, occupancy and utilisation diverge under a barrier, throttling is a clock staircase. Where a healthy feature distribution differs from simulation, the gap names missing physics.

3 · Fine-tune, then monitor

Replay diagnosed real incidents through the same windowing and fine-tune with simulation as pre-training. Keep the unsupervised score as an "unknown" class. Monitor drift on healthy-window features; use ML-versus-rules disagreement as the retraining trigger.

Known domain gaps

- Per-port asymmetry: real optics fail one at a time; the model gives siblings identical counters.
- Missing and jittered samples; a fleet's 1 Hz is not clean.
- Storage shares the fabric on many clusters; here it does not, and that separation is what makes phase change easy.
- Only six signatures. NIC flaps, HBM ECC storms, shared-aisle cooling, ECMP polarisation are unmodelled.

Roadmap, in order

- **Per-port telemetry asymmetry** in the simulator, then a port-level localiser.
- **More fault families** and mixed faults, so the classifier is multi-label.
- **Sequence model over windows** (or HMM smoothing) for latency and persistence instead of "two in a row".
- **Conformal thresholds per class** to give an honest false-alarm guarantee.
- **Incident replay harness**: one real diagnosed incident, scored as on slide 7.